

SmartEcoGymApp_Team_2.mlapp – Source Code

```
classdef SmartEcoGymApp_Team_2 < matlab.apps.AppBase

% Properties that correspond to app components
properties (Access = public)
    UIFigure                matlab.ui.Figure
    ForHRGraphListBox        matlab.ui.control.ListBox
    SessionsForHRAccelListBoxLabel matlab.ui.control.Label
    AlertSignal_1            matlab.ui.control.Image
    AlertSignal_2            matlab.ui.control.Image
    ImportantInsights        matlab.ui.control.TextArea
    YogaButton               matlab.ui.control.Button
    returnGenStats           matlab.ui.control.Button
    WalkingButton            matlab.ui.control.Button
    RunningButton            matlab.ui.control.Button
    EllipticalButton         matlab.ui.control.Button
    CyclingButton            matlab.ui.control.Button
    WeightliftingButton       matlab.ui.control.Button
    BodyweightButton         matlab.ui.control.Button
    Suggestions              matlab.ui.control.TextArea
    UniqueIDEditField        matlab.ui.control.EditField
    UniqueIDEditFieldLabel   matlab.ui.control.Label
    TitleBar_2               matlab.ui.control.Image
    Activity_Display         matlab.ui.control.Image
    User_Info                matlab.ui.control.TextArea
    TitleBar_1               matlab.ui.control.Image
    SmartEcoGymUserAnalyticsLabel matlab.ui.control.Label
    UserAnalytics            matlab.ui.control.TextArea
    HeartRateGraph           matlab.ui.control.UIAxes
    WorkoutEfficiencyGraph   matlab.ui.control.UIAxes
    CalBurnedVSPersonAvg     matlab.ui.control.UIAxes
end

properties (Access = private)
    peopleTable      % people data converted into table data type
    sessionsTable    % sessions data converted into table data type
    activityTable     % activity_MET_reference data converted into table data type
    fullSessionData  % the complete data table with each session also
                    % having the important activity and people data
                    % joined onto it

    currPersonID      % keeps track of the current Person ID entered in app
    currActivitySelect % keeps track of activity user selected on workout graphic
    chosenSessions     % keeps track of sessions chosen for HR / Accel Graph
end

methods (Access = private)

% Start of app's Helper Functions

% Most likely called when event occurs on the dashboard. All do
% not return and have 1 parameter, "app" which is the app itself
% and therefore gives access to the components and functions
% associated with it.

%-----

% Function: plotCalBurnedVAvg(app)
% Description: handles the plotting of CalBurnedVSPerson Component
%              by calculating avg cal burned and filtered by current
%              user and activity type.
function plotCalBurnedVAvg(app)

    % Gets rows of data only where both currPersonID and
    % currActivitySelect match
    if(strcmp(app.currActivitySelect, 'general'))
        personActivityData = app.fullSessionData(string(app.fullSessionData.person_id) ...
                                                    == string(app.currPersonID), :);
    else
        personActivityData = app.fullSessionData(string(app.fullSessionData.person_id) == string(app.currPersonID) & ...
                                                    string(app.fullSessionData.activity_type) == string(app.currActivitySelect), :);
    end

    % Calculates current user's avg cal burned per session,
    % filtered by the activity currently selected
    personAvtivityAvgCal = mean(personActivityData.calBurned, 'omitnan');

    % Plotting CalBurnedVSPerson Graph
    axes = app.CalBurnedVSPersonAvg;

    bar(axes, personActivityData.calBurned, 'FaceColor', [1.0000 0.4196 0.2902]);
    hold(axes, 'on');

    yline(axes, personAvtivityAvgCal, 'w--', 'LineWidth', 2, ...
        'Label', sprintf('Avg: %.1f kcal', personAvtivityAvgCal), ...
        'LabelVerticalAlignment', 'bottom');
    hold(axes, 'off');

    axes.XTick = 1:height(personActivityData);
    axes.XColor = [0.9294 0.9020 0.8549];
    axes.XTickLabel = string(personActivityData.session_id);
    axes.XTickLabelRotation = 45;
    axes.XLabel.String = 'Session ID';
    axes.YColor = [0.9294 0.9020 0.8549];
    axes.YLabel.String = 'Calories Burned (kcal)';
    axes.Title.Color = [0.9294 0.9020 0.8549];
    axes.Title.String = sprintf('Member %s (%s): Calories vs Personal Avg', ...
                                app.currPersonID, app.currActivitySelect);

    grid(axes, 'on');
end

% Function: plotHeartRate(app)
% Description: handles the plotting of HeartRateGraph Component by
%              filtering data by current user, activity, user's
%              chosen sessions and using resulting HR vectors to graph.
function plotHeartRate(app)

    % Gets rows of data only where both currPersonID and
```

```

% currActivitySelect match
if strcmp(app.currActivitySelect, 'general')
    personActivityData = app.fullSessionData(string(app.fullSessionData.person_id) ...
        == string(app.currPersonID), :);
else
    personActivityData = app.fullSessionData(string(app.fullSessionData.person_id) == string(app.currPersonID) & ...
        string(app.fullSessionData.activity_type) == string(app.currActivitySelect), :);
end

% Filter data to plot by the chosenSessions (Heart Rates can
% overcrowd the graph so users will select / multiselect which
% sessions they want to view).
personActivityData = personActivityData(ismember(string(personActivityData.session_id), ...
    string(app.chosenSessions)), :);

% Inside fullSessionData table, heart rate remains as
% cell arrays, so we must convert each one to number vectors to
% do operations on them and use them for graphs
numberedHR = cell2mat(personActivityData.heart_rate);
numberedTimes = cell2mat(personActivityData.time);

% Plotting HeartRateGraph
axes = app.HeartRateGraph;

plot(axes, numberedTimes, numberedHR, 'Color', [1.0000 0.4196 0.2902], 'LineWidth', 1.5);

axes.YLabel.String = 'Heart Rate (bpm)';
axes.XLabel.String = 'Workout Time (s)';
axes.Title.String = sprintf('Member %s (%s): Heart Rate vs. Time', ...
    app.currPersonID, app.currActivitySelect);
legend(axes, {'Heart Rate'});
end

% Function: writeToSessionsList(app)
% Description: updates ForHRGraphListBox by providing items /
%             sessions to choose from, filtered by person_id and
%             activity.
function writeToSessionsList(app)
    % Gets rows of data only where both currPersonID and
    % currActivitySelect match
    if strcmp(app.currActivitySelect, 'general')
        currSessions = app.sessionsTable(string(app.sessionsTable.person_id) == string(app.currPersonID), :);
    else
        currSessions = app.sessionsTable(string(app.sessionsTable.person_id) == string(app.currPersonID) & ...
            string(app.sessionsTable.activity_type) == string(app.currActivitySelect), :);
    end

    % Writes session IDs to Items in List Box Component
    app.ForHRGraphListBox.Items = cellstr(currSessions.session_id);
    app.ForHRGraphListBox.Visible = 'on';
end

% Function: plotWorkoutEfficiency(app)
% Description: handles the plotting of WorkoutEfficiencyGraph
%             component. Calculates efficiency in calBurned/hr for
%             each of the user's activities and plots them.
function plotWorkoutEfficiency(app)

    % Filters rows by current person id.
    currPersonSessions = app.fullSessionData(string(app.fullSessionData.person_id) == string(app.currPersonID), :);

    % Summarizing activity data by mean calBurned/hr
    activitySummary = groupsummary(currPersonSessions, 'activity_type', 'mean', 'calRate_per_hr');

    [~, sortIdx] = sort(activitySummary.mean_calRate_per_hr, 'descend');
    activitySummary = activitySummary(sortIdx, :);

    % Plotting WorkoutEfficiencyGraph
    axes = app.WorkoutEfficiencyGraph;

    % Horizontal bar plot for clear activity category comparison
    barh(axes, categorical(activitySummary.activity_type), activitySummary.mean_calRate_per_hr, 'FaceColor', [0.3098 0.8196 0.7686]);

    % Add numerical data labels on top of each bar
    text(axes, activitySummary.mean_calRate_per_hr + 10, 1:height(activitySummary), string(num2str(activitySummary.mean_calRate_per_hr, '%.1f kcal/hr')), ...
        'VerticalAlignment', 'middle', 'FontSize', 9, 'FontWeight', 'bold');

    axes.XLabel.String = 'Average Burn Rate (kcal / hour)';
    axes.YLabel.String = 'Workout Type';
    axes.Title.String = 'Your Workout Efficiency';
    grid(axes, 'on');

    % Extend x-axis limit so labels fit inside plot area
    axes.XLim = [0 max(activitySummary.mean_calRate_per_hr) * 1.25];
end

% Function: updateSuggestionsAndInsights(app)
% Description: handles updating the text in ImportantInsights and
%             Suggestions components using thresholds and user goals
%             that determine the user's success in workouts and
%             resulting advice.
function updateSuggestionsAndInsights(app)

    personActivityData = app.fullSessionData(string(app.fullSessionData.person_id) ...
        == string(app.currPersonID), :);

    % Find the current user's profile
    personRow = app.peopleTable(strcmp(app.peopleTable.person_id, app.currPersonID), :);
    goal = string(personRow.fitness_goal{1});
    averageCalories = mean(personActivityData.calBurned, "omitnan");
    averageDuration = mean(personActivityData.duration_hr, "omitnan") * 60;

    % Determine workout efficiency grading and updates alert signal
    % accordingly.
    if averageCalories >= 250 && averageDuration >= 30
        efficiencyMessage = "Excellent";
        app.AlertSignal_1.ImageSource = "excellentAlertSignal.png";
        app.AlertSignal_2.ImageSource = "excellentAlertSignal.png";
    elseif averageCalories >= 180 && averageDuration >= 20
        efficiencyMessage = "Good";
        app.AlertSignal_1.ImageSource = "goodAlertSignal.png";
    end

```

```

        app.AlertSignal_2.ImageSource = "goodAlertSignal.png";
    elseif averageCalories >= 120
        efficiencyMessage = "Fair";
        app.AlertSignal_1.ImageSource = "fairAlertSignal.png";
        app.AlertSignal_2.ImageSource = "fairAlertSignal.png";
    else
        efficiencyMessage = "Needs More Improvement";
        app.AlertSignal_1.ImageSource = "needsImprovementAlertSignal.png";
        app.AlertSignal_2.ImageSource = "needsImprovementAlertSignal.png";
    end

% Goal Specific Suggestions
switch goal
case "weight_loss"
    if averageCalories >= 250
        calorieSuggestion = "Looking at your average calories, you are currently meeting your calorie goal.";
    elseif averageCalories >= 180
        calorieSuggestion = "Looking at your average calories, you are close to hitting your calorie goal.";
    else
        calorieSuggestion = "Looking at your average calories, try increasing your workout duration.";
    end
    workoutSuggestion = "Recommended Workouts: Running, Cycling, Elliptical.";
    advice = "Advice: Add 8-10 minutes of cardio and increase your pace.";
case "strength"
    calorieSuggestion = "Looking at your average calories, we recommend focusing on the workout quality more than the calories burned.";
    workoutSuggestion = "Recommended Workouts: Weightlifting, Bodyweight Training.";
    advice = "Advice: Add another strength session.";
case "endurance"
    if averageDuration >= 30
        calorieSuggestion = "Looking at your average duration, your workout duration supports your endurance goal.";
    else
        calorieSuggestion = "Looking at your average duration, you should increase your workout time.";
    end
    workoutSuggestion = "Recommended Workouts: Running, Cycling, Elliptical.";
    advice = "Advice: Extend your next cardio session by 10 minutes.";
case "general_fitness"
    if averageCalories >= 200 && averageDuration >= 25
        calorieSuggestion = "Looking at your average calories and duration, you're maintaining a balanced workout routine.";
    elseif averageCalories >= 150
        calorieSuggestion = "Looking at your average calories and duration, you're making good progress. Try adding 5-10 minutes to your workouts.";
    else
        calorieSuggestion = "Looking at your average calories and duration, consider increasing workout duration or intensity.";
    end
    workoutSuggestion = "Recommended Workouts: Cardio, Strength Training, Yoga.";
    advice = "Advice: Aim for 200+ calories and 25-30 minute workouts during the week.";
otherwise
    workoutSuggestion = "Maintain a balance of cardio, strength, and good recovery.";
    calorieSuggestion = "Keep a consistent workout schedule.";
end

% Show the suggestions
app.Suggestions.Value = [
    "Suggestions"
    ""
    calorieSuggestion
    ""
    workoutSuggestion
    advice
];

% Show the important insights
app.ImportantInsights.Value = [
    "Important Insights"
    ""
    "Workout Efficiency: " + efficiencyMessage
    ""
    ""
    "Average Calories: " + sprintf("%.0f kcal", averageCalories)
    "Average Duration: " + sprintf("%.0f minutes", averageDuration)
];
end

%-----

% End of app's Helper Functions
end

% Callbacks that handle component events
methods (Access = private)

% Code that executes after component creation
function startupFcn(app)

    % Ensures required data file has been downloaded and correctly
    % pasted into the directory for the app to run.
    if ~isfile('final_generated_wearable_sensor_data.mat')
        uialert(app.UIFigure, ['Data file not found. Please download it from the google drive link in the README' ...
            ' and place it in the app folder.'], 'Missing Data File');
        return
    end

    % Loads data file at start of app
    rawData = load("final_generated_wearable_sensor_data.mat");
    app.currActivitySelect = 'general';

    % Converting raw data into correct Structs
    peopleStruct = [rawData.people{:}];
    sessionsStruct = [rawData.sessions{:}];
    activityStruct = [rawData.activity_met_reference{:}];

    % Converting Structs into tables
    app.peopleTable = struct2table(peopleStruct);
    app.sessionsTable = struct2table(sessionsStruct);
    app.activityTable = struct2table(activityStruct);

    % Joining peopleTable and activityTable onto sessionsTable to
    % create full session data
    app.fullSessionData = innerjoin(app.sessionsTable, app.peopleTable, 'Keys', 'person_id');
    app.fullSessionData = innerjoin(app.fullSessionData, app.activityTable, 'Keys', 'activity_type');

```

```

% Some important calculations will go here. These will be reused
% throughout the project, so we will only have to calculate them once:
% at the start of the program.

% Calculates duration of each workout and adds it to the table.
app.fullSessionData.duration_hr = (double(app.fullSessionData.end_time_sec) - ...
    double(app.fullSessionData.start_time_sec)) / 3600.0;

% Calculates calBurned of each workout and adds it to the table
% using the formula MET * weight * duration from GitHub guide.
app.fullSessionData.calBurned = app.fullSessionData.typical_MET .* app.fullSessionData.weight_kg .* ...
    double(app.fullSessionData.duration_hr);

% Calculates workout efficiency in calBurned per hr of each
% session and adds it to the table
app.fullSessionData.calRate_per_hr = app.fullSessionData.calBurned ./ app.fullSessionData.duration_hr;

end

% Value changed function: UniqueIDEditField
function UniqueIDEditFieldValueChanged(app, event)
    value = app.UniqueIDEditField.Value;

    % finding index where user entered value resides
    personIdx = strcmp(app.peopleTable.person_id, value);

    % First ensures that entered ID is a valid one.
    if ~any(personIdx)
        uialert(app.UIFigure, 'ID Does Not Exist. Try Again.', 'Error');
        app.UniqueIDEditField.Value = '';
        app.UniqueIDEditField.Placeholder = 'Try Again: Enter your ID';
    else
        % Sets current person ID to what user entered
        app.currPersonID = value;

        % Gets the row where person's ID index matches
        personRow = app.peopleTable(personIdx, :);

        % Writes user info from the row above into User Info Box
        userInfo = sprintf(['User Info \n\n\n\n\n' ...
            'Age:      %d years old\n' ...
            'Height:   %1f cm\n' ...
            'Weight:    %1f kg\n\n\n\n' ...
            'Fitness Goal: %s\n' ...
            'Fitness Level: %s\n\n\n\n' ...
            'Resting BPM: %4f bpm\n' ...
            'Max BPM:    %4f bpm\n' ...
            'BMR:      %0f kcal/day\n'], ...
            personRow.age_years, ...
            personRow.height_cm, ...
            personRow.weight_kg, ...
            personRow.fitness_goal{1}, ...
            personRow.fitness_level{1}, ...
            personRow.resting_hr_bpm, ...
            personRow.estimated_max_hr_bpm, ...
            personRow.bmr_kcal_day);

        app.User_Info.Value = userInfo;

        % On event user picks ID, we call helper functions to
        % update graphs and text components.
        plotCalBurnedVAvg(app);
        plotWorkoutEfficiency(app);
        plotHeartRate(app);

        app.CalBurnedVSPersonAvg.Visible = "on";
        app.WorkoutEfficiencyGraph.Visible = "on";
        app.HeartRateGraph.Visible = "on";

        writeToSessionsList(app);
        app.AlertSignal_1.Visible = "on";
        app.AlertSignal_2.Visible = "on";

        updateSuggestionsAndInsights(app);
    end
end

% Button pushed function: WalkingButton
function WalkingButtonPushed(app, event)
    app.currActivitySelect = 'walking';

    % Update plots given curr activity is now "walking"
    plotCalBurnedVAvg(app);
    writeToSessionsList(app);
    plotHeartRate(app);
end

% Button pushed function: returnGenStats
function returnGenStatsButtonPushed(app, event)
    app.currActivitySelect = 'general';

    % Update plots given curr activity is now "general"
    plotCalBurnedVAvg(app);
    writeToSessionsList(app);
    plotHeartRate(app);
end

% Button pushed function: RunningButton
function RunningButtonPushed(app, event)
    app.currActivitySelect = 'treadmill_running';

    % Update plots given curr activity is now "treadmill_running"
    plotCalBurnedVAvg(app);
    writeToSessionsList(app);
    plotHeartRate(app);
end

% Button pushed function: CyclingButton
function CyclingButtonPushed(app, event)
    app.currActivitySelect = 'cycling';

    % Update plots given curr activity is now "cycling"

```

```

        plotCalBurnedVAvg(app);
        writeToSessionsList(app);
        plotHeartRate(app);
    end

    % Button pushed function: EllipticalButton
    function EllipticalButtonPushed(app, event)
        app.currActivitySelect = 'elliptical';

        % Update plots given curr activity is now "elliptical"
        plotCalBurnedVAvg(app);
        writeToSessionsList(app);
        plotHeartRate(app);
    end

    % Button pushed function: WeightliftingButton
    function WeightliftingButtonPushed(app, event)
        app.currActivitySelect = 'weight_lifting';

        % Update plots given curr activity is now "weight_lifting"
        plotCalBurnedVAvg(app);
        writeToSessionsList(app);
        plotHeartRate(app);
    end

    % Button pushed function: BodyweightButton
    function BodyweightButtonPushed(app, event)
        app.currActivitySelect = 'bodyweight_training';

        % Update plots given curr activity is now "bodyweight_training"
        plotCalBurnedVAvg(app);
        writeToSessionsList(app);
        plotHeartRate(app);
    end

    % Button pushed function: YogaButton
    function YogaButtonPushed(app, event)
        app.currActivitySelect = 'yoga';

        % Update plots given curr activity is now "yoga"
        plotCalBurnedVAvg(app);
        writeToSessionsList(app);
        plotHeartRate(app);
    end

    % Value changed function: ForHRGraphListBox
    function ForHRGraphListBoxValueChanged(app, event)
        app.chosenSessions = app.ForHRGraphListBox.Value;

        % Update HR graph given the user has changed the sessions they
        % wish to view
        plotHeartRate(app);
    end
end

% Component initialization
methods (Access = private)

    % Create UIFigure and components
    function createComponents(app)

        % Get the file path for locating images
        pathToMLAPP = fileparts(mfilename('fullpath'));

        % Create UIFigure and hide until all components are created
        app.UIFigure = uifigure('Visible', 'off');
        app.UIFigure.Color = [0.1216 0.1608 0.149];
        app.UIFigure.Position = [1 1 1439 879];
        app.UIFigure.Name = 'MATLAB App';
        app.UIFigure.Resize = 'off';
        app.UIFigure.Theme = 'dark';
        app.UIFigure.WindowStyle = 'alwaysontop';

        % Create CalBurnedVSPersonAvg
        app.CalBurnedVSPersonAvg = uiaxes(app.UIFigure);
        title(app.CalBurnedVSPersonAvg, 'Title')
        xlabel(app.CalBurnedVSPersonAvg, 'X')
        ylabel(app.CalBurnedVSPersonAvg, 'Y')
        zlabel(app.CalBurnedVSPersonAvg, 'Z')
        app.CalBurnedVSPersonAvg.Visible = 'off';
        app.CalBurnedVSPersonAvg.Position = [901 496 478 236];

        % Create WorkoutEfficiencyGraph
        app.WorkoutEfficiencyGraph = uiaxes(app.UIFigure);
        title(app.WorkoutEfficiencyGraph, 'Title')
        xlabel(app.WorkoutEfficiencyGraph, 'X')
        ylabel(app.WorkoutEfficiencyGraph, 'Y')
        zlabel(app.WorkoutEfficiencyGraph, 'Z')
        app.WorkoutEfficiencyGraph.Visible = 'off';
        app.WorkoutEfficiencyGraph.Position = [905 13 474 226];

        % Create HeartRateGraph
        app.HeartRateGraph = uiaxes(app.UIFigure);
        title(app.HeartRateGraph, 'Title')
        xlabel(app.HeartRateGraph, 'X')
        ylabel(app.HeartRateGraph, 'Y')
        zlabel(app.HeartRateGraph, 'Z')
        app.HeartRateGraph.Visible = 'off';
        app.HeartRateGraph.Position = [962 238 470 259];

        % Create UserAnalytics
        app.UserAnalytics = uitextarea(app.UIFigure);
        app.UserAnalytics.Editable = 'off';
        app.UserAnalytics.HorizontalAlignment = 'center';
        app.UserAnalytics.FontName = 'Cambria';
        app.UserAnalytics.FontSize = 18;
        app.UserAnalytics.FontColor = [0.9294 0.902 0.851];
        app.UserAnalytics.BackgroundColor = [0.3608 0.4196 0.3686];
        app.UserAnalytics.Position = [920 732 500 45];
        app.UserAnalytics.Value = {'User Analytics'; ''};

        % Create SmartEcoGymUserAnalyticsLabel

```

```

app.SmartEcoGymUserAnalyticsLabel = uilabel(app.UIFigure);
app.SmartEcoGymUserAnalyticsLabel.FontName = 'Rockwell';
app.SmartEcoGymUserAnalyticsLabel.FontSize = 36;
app.SmartEcoGymUserAnalyticsLabel.FontColor = [0.9294 0.902 0.851];
app.SmartEcoGymUserAnalyticsLabel.Position = [60 812 608 58];
app.SmartEcoGymUserAnalyticsLabel.Text = 'Smart Eco Gym User Analytics';

% Create TitleBar_1
app.TitleBar_1 = uimage(app.UIFigure);
app.TitleBar_1.Position = [10 797 921 17];
app.TitleBar_1.ImageSource = fullfile(pathToMLAPP, 'ecogymLine.png');

% Create User_Info
app.User_Info = uitextarea(app.UIFigure);
app.User_Info.Editable = 'off';
app.User_Info.HorizontalAlignment = 'center';
app.User_Info.FontName = 'Cambria';
app.User_Info.FontSize = 18;
app.User_Info.FontColor = [0.9294 0.902 0.851];
app.User_Info.BackgroundColor = [0.3608 0.4196 0.3686];
app.User_Info.Position = [9 287 280 490];
app.User_Info.Value = {'User Info'};

% Create Activity_Display
app.Activity_Display = uimage(app.UIFigure);
app.Activity_Display.Position = [303 193 539 678];
app.Activity_Display.ImageSource = fullfile(pathToMLAPP, 'smartEcoGymActivity.png');

% Create TitleBar_2
app.TitleBar_2 = uimage(app.UIFigure);
app.TitleBar_2.Position = [570 797 871 17];
app.TitleBar_2.ImageSource = fullfile(pathToMLAPP, 'ecogymLine.png');

% Create UniqueIDEditFieldLabel
app.UniqueIDEditFieldLabel = uilabel(app.UIFigure);
app.UniqueIDEditFieldLabel.HorizontalAlignment = 'right';
app.UniqueIDEditFieldLabel.Position = [26 710 62 22];
app.UniqueIDEditFieldLabel.Text = 'Unique ID:';

% Create UniqueIDEditField
app.UniqueIDEditField = uieditfield(app.UIFigure, 'text');
app.UniqueIDEditField.ValueChangedFcn = createCallbackFcn(app, @UniqueIDEditFieldValueChanged, true);
app.UniqueIDEditField.Placeholder = 'Enter Your Anonymous ID';
app.UniqueIDEditField.Position = [103 710 158 22];

% Create Suggestions
app.Suggestions = uitextarea(app.UIFigure);
app.Suggestions.Editable = 'off';
app.Suggestions.HorizontalAlignment = 'center';
app.Suggestions.FontName = 'Cambria';
app.Suggestions.FontSize = 18;
app.Suggestions.FontColor = [0.9294 0.902 0.851];
app.Suggestions.BackgroundColor = [0.3608 0.4196 0.3686];
app.Suggestions.Position = [396 34 490 218];
app.Suggestions.Value = {'Suggestions'; ''};

% Create BodyweightButton
app.BodyweightButton = uibutton(app.UIFigure, 'push');
app.BodyweightButton.ButtonPushedFcn = createCallbackFcn(app, @BodyweightButtonPushed, true);
app.BodyweightButton.BackgroundColor = [0.7686 0.5333 0.302];
app.BodyweightButton.Position = [521 410 79 20];
app.BodyweightButton.Text = 'Click Me';

% Create WeightliftingButton
app.WeightliftingButton = uibutton(app.UIFigure, 'push');
app.WeightliftingButton.ButtonPushedFcn = createCallbackFcn(app, @WeightliftingButtonPushed, true);
app.WeightliftingButton.BackgroundColor = [0.7686 0.5333 0.302];
app.WeightliftingButton.Position = [381 390 79 20];
app.WeightliftingButton.Text = 'Click Me';

% Create CyclingButton
app.CyclingButton = uibutton(app.UIFigure, 'push');
app.CyclingButton.ButtonPushedFcn = createCallbackFcn(app, @CyclingButtonPushed, true);
app.CyclingButton.BackgroundColor = [0.7686 0.5333 0.302];
app.CyclingButton.Position = [651 520 79 20];
app.CyclingButton.Text = 'Click Me';

% Create EllipticalButton
app.EllipticalButton = uibutton(app.UIFigure, 'push');
app.EllipticalButton.ButtonPushedFcn = createCallbackFcn(app, @EllipticalButtonPushed, true);
app.EllipticalButton.BackgroundColor = [0.7686 0.5333 0.302];
app.EllipticalButton.Position = [651 710 79 20];
app.EllipticalButton.Text = 'Click Me';

% Create RunningButton
app.RunningButton = uibutton(app.UIFigure, 'push');
app.RunningButton.ButtonPushedFcn = createCallbackFcn(app, @RunningButtonPushed, true);
app.RunningButton.BackgroundColor = [0.7686 0.5333 0.302];
app.RunningButton.Position = [481 600 79 20];
app.RunningButton.Text = 'Click Me';

% Create WalkingButton
app.WalkingButton = uibutton(app.UIFigure, 'push');
app.WalkingButton.ButtonPushedFcn = createCallbackFcn(app, @WalkingButtonPushed, true);
app.WalkingButton.BackgroundColor = [0.7686 0.5294 0.302];
app.WalkingButton.Position = [381 600 79 20];
app.WalkingButton.Text = 'Click Me';

% Create returnGenStats
app.returnGenStats = uibutton(app.UIFigure, 'push');
app.returnGenStats.ButtonPushedFcn = createCallbackFcn(app, @returnGenStatsButtonPushed, true);
app.returnGenStats.BackgroundColor = [0.7686 0.5294 0.302];
app.returnGenStats.Position = [561 337 240 23];
app.returnGenStats.Text = 'Return To General Stats';

% Create YogaButton
app.YogaButton = uibutton(app.UIFigure, 'push');
app.YogaButton.ButtonPushedFcn = createCallbackFcn(app, @YogaButtonPushed, true);
app.YogaButton.BackgroundColor = [0.7686 0.5333 0.302];
app.YogaButton.Position = [701 390 79 20];
app.YogaButton.Text = 'Click Me';

```

```

% Create ImportantInsights
app.ImportantInsights = uitextarea(app.UIFigure);
app.ImportantInsights.Editable = 'off';
app.ImportantInsights.HorizontalAlignment = 'center';
app.ImportantInsights.FontName = 'Cambria';
app.ImportantInsights.FontSize = 18;
app.ImportantInsights.FontColor = [0.9294 0.902 0.851];
app.ImportantInsights.BackgroundColor = [0.3608 0.4196 0.3686];
app.ImportantInsights.Position = [11 34 368 218];
app.ImportantInsights.Value = {'Important Insights'; ''};

% Create AlertSignal_2
app.AlertSignal_2 = uimage(app.UIFigure);
app.AlertSignal_2.Visible = 'off';
app.AlertSignal_2.Position = [311 166 62 64];

% Create AlertSignal_1
app.AlertSignal_1 = uimage(app.UIFigure);
app.AlertSignal_1.Visible = 'off';
app.AlertSignal_1.Position = [21 166 62 64];

% Create SessionsForHRAccellListBoxLabel
app.SessionsForHRAccellListBoxLabel = uilabel(app.UIFigure);
app.SessionsForHRAccellListBoxLabel.HorizontalAlignment = 'right';
app.SessionsForHRAccellListBoxLabel.Position = [841 454 134 43];
app.SessionsForHRAccellListBoxLabel.Text = 'For HR / Accel Graph';

% Create ForHRGraphListBox
app.ForHRGraphListBox = uilistbox(app.UIFigure);
app.ForHRGraphListBox.Multiselect = 'on';
app.ForHRGraphListBox.ValueChangedFcn = createCallbackFcn(app, @ForHRGraphListBoxValueChanged, true);
app.ForHRGraphListBox.Visible = 'off';
app.ForHRGraphListBox.Position = [858 317 100 138];
app.ForHRGraphListBox.Value = {'Item 1'};

% Show the figure after all components are created
app.UIFigure.Visible = 'on';
end
end

% App creation and deletion
methods (Access = public)

% Construct app
function app = SmartEcoGymApp_Team_2

% Create UIFigure and components
createComponents(app)

% Register the app with App Designer
registerApp(app, app.UIFigure)

% Execute the startup function
runStartupFcn(app, @startupFcn)

if nargin == 0
clear app
end
end

% Code that executes before app deletion
function delete(app)

% Delete UIFigure when app is deleted
delete(app.UIFigure)
end
end
end
end

```